

Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

ShopEase SQL Assignment — Complete Solutions

Field	Details
Intern Name	Bharat
Organization	Celebal Technologies
Task	Week 2 — E-Commerce Sales Database
Database	ShopEase (MySQL / SQLite compatible)
Tables	customers, products, orders, order_items
Sections	A (SQL Basics) · B (Filtering) · C (Aggregation) · D (Joins) · E (Advanced)
Total Questions	27
Date	27 May 2026

Section A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Display all columns and rows from the customers table.

SQL Query:

```
SELECT * FROM customers;
```

Output:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Result: All 8 customer records are retrieved with every column.

Q2. Retrieve only first_name, last_name, and city of all customers.

SQL Query:

```
SELECT first_name, last_name, city FROM customers;
```

Output:

first_name	last_name	city
Aarav	Sharma	Mumbai
Priya	Patel	Ahmedabad
Rohan	Gupta	Delhi
Sneha	Reddy	Hyderabad
Vikram	Singh	Jaipur
Ananya	Iyer	Chennai
Karan	Mehta	Pune
Divya	Nair	Kochi

Q3. List all unique categories in the products table.

SQL Query:

```
SELECT DISTINCT category FROM products;
```

Output:

category
Electronics
Clothing
Home

Result: ShopEase sells 3 distinct product categories — Electronics, Clothing, and Home.

Q4. Identify the Primary Key of each table. Explain why a PK must be UNIQUE and NOT NULL.

Table	Primary Key	Data Type
customers	customer_id	INT
products	product_id	INT
orders	order_id	INT
order_items	item_id	INT

Why UNIQUE: A Primary Key uniquely identifies each record in a table. If two rows had the same PK value, the database could not distinguish between them, leading to data corruption and incorrect query results. For example, two customers with `customer_id = 101` would make it impossible to know whose order belongs to whom.

Why NOT NULL: A NULL value means 'unknown' or 'missing'. If a PK could be NULL, a row would have no identity — it couldn't be referenced by foreign keys, updated precisely, or deleted reliably. The database enforces NOT NULL on PKs automatically to guarantee every row has a definite, findable identity.

Q5. What constraints are on the email column in customers? What happens on duplicate email insert?

The **email** column in the customers table has two constraints:

1. **UNIQUE** — no two customers can share the same email address.
2. **NOT NULL** — every customer must provide an email address.

Duplicate insert attempt:

```
INSERT INTO customers VALUES (109, 'Test', 'User', 'aarav.s@email.com', 'Delhi', 'Delhi', '2024-09-01', FALSE);
```

Error: UNIQUE constraint failed: customers.email

Since 'aarav.s@email.com' already belongs to customer 101 (Aarav Sharma), the database rejects this insert entirely. The UNIQUE constraint acts as a guard against data duplication — ensuring one account per email address, just like any real e-commerce platform.

Q6. Try inserting a product with unit_price = -50. What happens?

INSERT attempt:

```
INSERT INTO products VALUES (209, 'Broken Item', 'Electronics', 'NoName', -50.00, 100);
```

Error: CHECK constraint failed: unit_price > 0

The products table has the constraint CHECK (`unit_price > 0`) defined on the `unit_price` column. A price of -50 violates this rule because prices must be positive. The database engine evaluates $(-50 > 0) = \text{FALSE}$ and immediately rejects the INSERT. This prevents logically invalid data (negative prices) from ever entering the database, ensuring business rules are enforced at the storage layer itself.

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

```
SELECT * FROM orders WHERE status = 'Delivered';
```

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498
1002	102	2024-08-03	Delivered	799
1004	101	2024-08-10	Delivered	3499
1006	105	2024-08-15	Delivered	5898
1008	103	2024-08-20	Delivered	899
1010	108	2024-08-28	Delivered	1598

Result: 6 out of 10 orders have been successfully delivered, totalling ₹17,191.

Q8. Find all Electronics products with unit_price > ₹2000.

```
SELECT product_id, product_name, category, brand, unit_price FROM products WHERE category = 'Electronics' AND unit_price > 2000;
```

product_id	product_name	category	brand	unit_price
203	Smart Watch	Electronics	Noise	2999
205	Bluetooth Speaker	Electronics	JBL	3499

Result: Smart Watch (₹2999) and Bluetooth Speaker (₹3499) qualify — premium electronics.

Q9. List customers who joined in 2024 and belong to Maharashtra.

```
SELECT customer_id, first_name, last_name, city, state, join_date FROM customers WHERE strftime('%Y', join_date) = '2024' AND state = 'Maharashtra'; -- MySQL equivalent: -- WHERE YEAR(join_date) = 2024 AND state = 'Maharashtra';
```

customer_id	first_name	last_name	city	state	join_date
101	Aarav	Sharma	Mumbai	Maharashtra	2024-01-15
107	Karan	Mehta	Pune	Maharashtra	2024-07-22

Result: Aarav Sharma (Mumbai) and Karan Mehta (Pune) — both Maharashtra customers from 2024.

Q10. Orders between 2024-08-10 and 2024-08-25 (inclusive) that are NOT Cancelled.

```
SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status != 'Cancelled';
```

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499
1006	105	2024-08-15	Delivered	5898
1007	106	2024-08-18	Pending	1299
1008	103	2024-08-20	Delivered	899
1009	107	2024-08-25	Shipped	6098

Result: 5 orders found. Note: Order 1005 (Sneha Reddy, Cancelled on Aug 12) is correctly excluded.

Q11. Explain `idx_orders_date`. How does it improve date-filter query performance?

What the index does: `CREATE INDEX idx_orders_date ON orders(order_date)` builds a separate, sorted B-Tree data structure on the `order_date` column. This allows the database engine to binary-search for a date range instead of scanning every row sequentially.

Without index: Full Table Scan — the engine reads all 10 (or 10 million) rows one by one to check each date. Time complexity: $O(n)$.

With index: Index Range Scan — the engine jumps directly to the first matching date in the B-Tree and reads only qualifying rows. Time complexity: $O(\log n + k)$, where k = matching rows.

Query that directly benefits:

```
SELECT order_id, customer_id, status, total_amount FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'; -- The optimizer uses idx_orders_date to seek directly to Aug 10, -- then scans forward to Aug 25 – no full table scan needed.
```

Q12. Would `YEAR(join_date) = 2024` use the `join_date` index? Rewrite as SARGable.

Original query (NOT SARGable):

```
SELECT * FROM customers WHERE YEAR(join_date) = 2024;
```

Why the index is NOT used: Applying the `YEAR()` function to `join_date` wraps the column inside a function call. The index is built on the raw `join_date` values, not on `YEAR(join_date)`. The query optimizer cannot use a B-Tree index to satisfy `YEAR(join_date) = 2024` without evaluating every row — so it falls back to a full table scan.

SARGable rewrite (index-friendly):

```
SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date < '2025-01-01'; -- Now join_date is on the left side with no function wrapping. -- The index idx_customers on join_date (if it existed) can be used -- for an efficient range scan between Jan 1 and Dec 31 2024.
```

SARGable (Search ARGument ABLE) means the predicate can use an index seek. The rule: never apply a function to an indexed column in a WHERE clause — instead, transform the constant on the right-hand side.

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders.

```
SELECT COUNT(*) AS total_orders FROM orders;
```

total_orders
10

Result: ShopEase has 10 total orders in the database.

Q14. Find total revenue from all 'Delivered' orders.

```
SELECT ROUND(SUM(total_amount), 2) AS total_revenue FROM orders WHERE status = 'Delivered';
```

total_revenue
17191.0

Result: ■17,191 total revenue successfully collected from 6 delivered orders.

Q15. Calculate the average unit_price of products in each category.

```
SELECT category, ROUND(AVG(unit_price), 2) AS avg_unit_price FROM products GROUP BY category;
```

category	avg_unit_price
Clothing	2699.0
Electronics	2224.0
Home	949.0

Result: Clothing has the highest average price (■2699 — driven by Nike Running Shoes ■4599). Home category is the most affordable at ■949 average.

Q16. For each order status, find count of orders and total revenue. Sort by revenue DESC.

```
SELECT status, COUNT(*) AS order_count, ROUND(SUM(total_amount), 2) AS total_revenue FROM orders GROUP BY status ORDER BY total_revenue DESC;
```

status	order_count	total_revenue
Delivered	6	17191.0
Shipped	2	13596.0
Cancelled	1	2999.0
Pending	1	1299.0

Result: Delivered orders lead with ■17,191 revenue. Shipped orders (■13,596) are pending completion. The single Cancelled order represents ■2,999 in lost revenue.

Q17. Find the most expensive and cheapest product in each category.

```
SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest FROM products GROUP BY category;
```

category	most_expensive	cheapest
Clothing	4599	799

category	most_expensive	cheapest
Electronics	3499	899
Home	1299	599

Q18. List categories where average unit_price > 2000. (HAVING clause)

```
SELECT category, ROUND(AVG(unit_price), 2) AS avg_price FROM products GROUP BY category
HAVING AVG(unit_price) > 2000;
```

category	avg_price
Clothing	2699.0
Electronics	2224.0

Result: Clothing (2699) and Electronics (2224) exceed the 2000 average threshold. Home products (949 avg) do not qualify. Note: HAVING filters after GROUP BY aggregation, whereas WHERE filters individual rows before grouping.

Section D — Joins & Relationships

Q19. INNER JOIN — each order with customer's first_name and last_name.

```
SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498
1002	2024-08-03	Priya	Patel	799
1003	2024-08-05	Rohan	Gupta	7498
1004	2024-08-10	Aarav	Sharma	3499
1005	2024-08-12	Sneha	Reddy	2999
1006	2024-08-15	Vikram	Singh	5898
1007	2024-08-18	Ananya	Iyer	1299
1008	2024-08-20	Rohan	Gupta	899
1009	2024-08-25	Karan	Mehta	6098
1010	2024-08-28	Divya	Nair	1598

Result: All 10 orders matched with their customer names. INNER JOIN returns only rows where a match exists in BOTH tables — here every order has a valid customer.

Q20. LEFT JOIN — ALL customers and their orders (NULL if no orders).

```
SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date, o.status,
o.total_amount FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

customer_id	first_name	last_name	order_id	order_date	status	total_amount
101	Aarav	Sharma	1001	2024-08-01	Delivered	4498
101	Aarav	Sharma	1004	2024-08-10	Delivered	3499
102	Priya	Patel	1002	2024-08-03	Delivered	799
103	Rohan	Gupta	1003	2024-08-05	Shipped	7498
103	Rohan	Gupta	1008	2024-08-20	Delivered	899
104	Sneha	Reddy	1005	2024-08-12	Cancelled	2999
105	Vikram	Singh	1006	2024-08-15	Delivered	5898
106	Ananya	Iyer	1007	2024-08-18	Pending	1299
107	Karan	Mehta	1009	2024-08-25	Shipped	6098
108	Divya	Nair	1010	2024-08-28	Delivered	1598

Result: All 8 customers appear. In this dataset every customer has at least one order, so no NULL rows appear. A customer who had never ordered would show NULL in all order columns — that is the LEFT JOIN guarantee: every left-table row is preserved.

Q21. Three-table JOIN: orders → order_items → products.

```
SELECT o.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct FROM orders
o JOIN order_items oi ON o.order_id = oi.order_id JOIN products p ON oi.product_id =
p.product_id ORDER BY o.order_id;
```


order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499	0
1001	Laptop Stand	1	899	10
1002	Cotton T-Shirt	1	799	0
1003	Smart Watch	1	2999	0
1003	Running Shoes	1	4599	5
1004	Bluetooth Speaker	1	3499	0
1005	Smart Watch	1	2999	0
1006	Wireless Earbuds	1	1499	10
1006	Running Shoes	1	4599	5
1007	Bedsheet Set	1	1299	0
1008	Laptop Stand	1	899	0
1009	Bluetooth Speaker	1	3499	0
1009	Cushion Covers (Set)	2	599	15
1010	Bedsheet Set	1	1299	0
1010	Cushion Covers (Set)	1	599	0

Result: 15 line items across 10 orders. The three-table join traverses the full relationship chain: customers → orders → order_items → products.

Q22. Difference between LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

LEFT JOIN: Returns ALL rows from the left table and matching rows from the right table. Where no match exists in the right table, NULLs fill the right columns.

Example: customers LEFT JOIN orders — shows every customer, even those without orders.

RIGHT JOIN: Returns ALL rows from the right table and matching rows from the left table. Where no match exists in the left table, NULLs fill the left columns.

Example: customers RIGHT JOIN orders — shows every order, even if the customer record was somehow deleted (orphaned orders).

FULL OUTER JOIN: Returns ALL rows from BOTH tables. Where no match exists on either side, NULLs fill the missing side's columns. This is the union of LEFT and RIGHT JOINS.

When to use: Use FULL OUTER JOIN when you need to audit both sides simultaneously — e.g., finding customers with no orders AND orders with no customers in one query. Note: MySQL does not natively support FULL OUTER JOIN; it is emulated with UNION of LEFT and RIGHT JOINS.

Q23. Identify all Foreign Key relationships. What happens with customer_id = 999?

Child Table	Foreign Key Column	Parent Table	Referenced Column
orders	customer_id	customers	customer_id
order_items	order_id	orders	order_id
order_items	product_id	products	product_id

```
INSERT INTO orders VALUES (1011, 999, '2024-09-01', 'Pending', 500.00);
```

Error: FOREIGN KEY constraint failed — customer_id = 999 does not exist in the customers table.

Referential integrity enforcement means the database will reject any row in a child table that references a non-existent primary key in the parent table. This prevents orphaned records and ensures every order is always traceable back to a real customer.

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. CASE statement — classify products into price tiers.

```
SELECT product_name, unit_price, CASE WHEN unit_price < 1000 THEN 'Budget' WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range' ELSE 'Premium' END AS price_tier FROM products ORDER BY unit_price;
```

product_name	unit_price	price_tier
Cushion Covers (Set)	599	Budget
Cotton T-Shirt	799	Budget
Laptop Stand	899	Budget
Bedsheet Set	1299	Mid-Range
Wireless Earbuds	1499	Mid-Range
Smart Watch	2999	Mid-Range
Bluetooth Speaker	3499	Premium
Running Shoes	4599	Premium

Result: 3 Budget products (under 1000), 3 Mid-Range (1000-3000), 2 Premium (above 3000 — Bluetooth Speaker and Running Shoes).

Q25. CASE inside aggregate — count Delivered vs Not Delivered in a single row.

```
SELECT SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered, SUM(CASE WHEN status != 'Delivered' THEN 1 ELSE 0 END) AS not_delivered FROM orders;
```

delivered	not_delivered
6	4

Result: 6 delivered, 4 not delivered (2 Shipped + 1 Pending + 1 Cancelled). This conditional aggregation technique avoids the need for multiple queries or subqueries.

Q26. Explain each letter of ACID with a real-world bank transfer example.

Property	Full Name	Definition	Bank Transfer Example
A	Atomicity	A transaction is ALL-or-NOTHING. Either everything succeeds or nothing does.	Depositing Rs. 5000 into a savings account. If the debit fails, the credit is not applied.
C	Consistency	A transaction brings the database from one valid state to another valid state.	Transferring Rs. 5000 from a checking account to a savings account. The total balance remains the same.
I	Isolation	Concurrent transactions execute as if they were sequential.	Two people trying to withdraw money from the same ATM at the same time. The system ensures one completes first.
D	Durability	Once a transaction is committed, it is permanent.	After a successful transfer, the bank's ledger is updated and the transaction is irreversible.

Q27. Complete SQL Transaction — atomically insert order, items, and update stock.

The transaction below does all 4 steps atomically. If any step fails, the entire block is rolled back and no partial changes persist in the database.

```
-- ===== -- TRANSACTION: Insert new order 1011 with two items -- and update product stock quantities atomically --
===== START TRANSACTION; --
MySQL/PostgreSQL -- (Use BEGIN; in PostgreSQL or SQLite) -- STEP 1: Insert the new order
INSERT INTO orders (order_id, customer_id, order_date, status, total_amount) VALUES (1011,
```

```

102, CURDATE(), 'Pending', 1598.00); -- STEP 2a: Insert first order item (Wireless Earbuds
x1) INSERT INTO order_items (item_id, order_id, product_id, quantity, unit_price,
discount_pct) VALUES (5016, 1011, 201, 1, 1499.00, 0); -- STEP 2b: Insert second order
item (Cushion Covers x1) INSERT INTO order_items (item_id, order_id, product_id, quantity,
unit_price, discount_pct) VALUES (5017, 1011, 208, 1, 599.00, 0); -- STEP 3a: Reduce stock
for Wireless Earbuds (product 201) by 1 UPDATE products SET stock_qty = stock_qty - 1
WHERE product_id = 201; -- STEP 3b: Reduce stock for Cushion Covers (product 208) by 1
UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208; -- STEP 4: If all
steps succeeded, commit permanently COMMIT; --
===== -- ERROR HANDLER (Stored
Procedure / Application layer) -- In MySQL use DECLARE EXIT HANDLER FOR SQLEXCEPTION
ROLLBACK; -- ===== -- MySQL Stored
Procedure version with full error handling: DELIMITER $$ CREATE PROCEDURE
place_order_1011() BEGIN DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; SELECT
'Transaction failed – all changes rolled back.' AS error_msg; END; START TRANSACTION;
INSERT INTO orders VALUES (1011, 102, CURDATE(), 'Pending', 1598.00); INSERT INTO
order_items VALUES (5016, 1011, 201, 1, 1499.00, 0); INSERT INTO order_items VALUES (5017,
1011, 208, 1, 599.00, 0); UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id =
201; UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208; COMMIT; SELECT
'Transaction committed successfully.' AS status_msg; END$$ DELIMITER ; -- Execute: CALL
place_order_1011();

```

Atomicity guarantee: The EXIT HANDLER catches any SQL exception (constraint violation, deadlock, connection error) and executes ROLLBACK, undoing all changes from the START TRANSACTION. Either ALL 5 statements succeed and COMMIT is reached, or NONE of the changes persist. This prevents partial inserts such as an order existing without its items, or stock being reduced without a corresponding order being recorded.

Assignment Summary

Section	Questions	Topics Covered
A — SQL Basics	Q1–Q6	SELECT *, column selection, DISTINCT, Primary Keys, UNIQUE, NOT NULL, CHECK constraints
B — Filtering	Q7–Q12	WHERE, BETWEEN, AND/!=, Index types, SARGable queries, idx_orders_date
C — Aggregation	Q13–Q18	COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING
D — Joins	Q19–Q23	INNER JOIN, LEFT JOIN, 3-table JOIN, LEFT vs RIGHT vs FULL OUTER, FK constraints
E — Advanced	Q24–Q27	CASE expressions, conditional aggregation, ACID properties, BEGIN/COMMIT/ROLLBACK transactions

All 27 questions have been answered with working SQL queries verified against the ShopEase sample database. Every query output shown is the actual result from executing against the provided INSERT data. Queries are written in standard SQL compatible with MySQL, PostgreSQL, and SQLite.